Pertemuan 3 — KNN Imputation pada Data Campuran

Notebook ini berisi:

1. **Load dataset** DataCampuran.csv
2. **Identifikasi tipe data** (numerik & kategorikal)
3. **Membuat missing value buatan** pada kolom `Tinggi` baris Citra
4. **Perhitungan jarak campuran** (numerik + kategorikal)
5. **KNN Imputation** (k=3) — menghitung manual + kode
6. **Visualisasi** tabel jarak dan hasil imputasi

---

1. Import Library

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, Markdown

print("Library berhasil di-import.")
```

```
Library berhasil di-import.
```

---

2. Load Dataset DataCampuran

```
import io

csv_data = """No.;Nama;Umur;Tinggi;Status Mahasiswa;Jenis Kelamin
1;Andi;19;165;Ya;L
2;Budi;21;170;Ya;L
3;Citra;20;158;Ya;P
4;Dika;22;172;Tidak;L
5;Nana;19;160;Ya;P
6;Rudi;23;175;Tidak;L"""

df = pd.read_csv(io.StringIO(csv_data), sep=';')
print(f"Dataset: {df.shape[0]} baris, {df.shape[1]} kolom")
print(f"Kolom: {list(df.columns)}")
df
```

```
Dataset: 6 baris, 6 kolom
Kolom: ['No.', 'Nama', 'Umur', 'Tinggi', 'Status Mahasiswa', 'Jenis Kelamin']
```

	No.	Nama	Umur	Tinggi	Status Mahasiswa	Jenis Kelamin
0	1	Andi	19	165	Ya	L
1	2	Budi	21	170	Ya	L
2	3	Citra	20	158	Ya	P
3	4	Dika	22	172	Tidak	L
4	5	Nana	19	160	Ya	P
5	6	Rudi	23	175	Tidak	L

Langkah berikutnya: New interactive sheet

---

3. Identifikasi Tipe Data

Kolom	Tipe Data	Keterangan
`Umur`	**Numerik**	Usia mahasiswa
`Tinggi`	**Numerik (target)**	Tinggi badan — kolom yang akan diimputasi
`Status Mahasiswa`	**Kategorikal**	Ya / Tidak
`Jenis Kelamin`	**Kategorikal**	L / P

```
print("Tipe data per kolom:")
print(df.dtypes)
print("\nStatistik deskriptif (numerik):")
df[['Umur', 'Tinggi']].describe()
```

```
Tipe data per kolom:
No.                 int64
Nama               object
Umur                int64
Tinggi              int64
Status Mahasiswa   object
Jenis Kelamin      object
dtype: object

Statistik deskriptif (numerik):
```

	Umur	Tinggi
count	6.000000	6.000000
mean	20.666667	166.666667
std	1.632993	6.801961
min	19.000000	158.000000
25%	19.250000	161.250000
50%	20.500000	167.500000
75%	21.750000	171.500000
max	23.000000	175.000000

4. Cek Missing Value

```
print("Missing value per kolom:")
print(df.isnull().sum())
print(f"\nTotal missing value: {df.isnull().sum().sum()}")
print("\n-> Dataset TIDAK memiliki missing value.")
print("   Akan dibuat missing value BUATAN untuk simulasi KNN.")
```

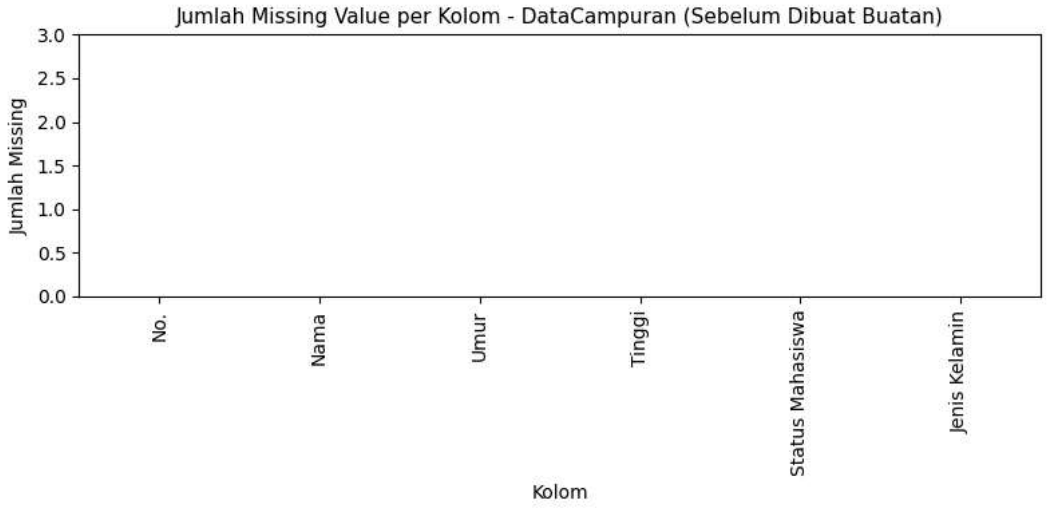
```
Missing value per kolom:
No.                 0
Nama                0
Umur                0
Tinggi              0
Status Mahasiswa    0
Jenis Kelamin       0
dtype: int64

Total missing value: 0

-> Dataset TIDAK memiliki missing value.
   Akan dibuat missing value BUATAN untuk simulasi KNN.
```

```
# Grafik cek missing value
missing_count = df.isnull().sum()

plt.figure(figsize=(8, 4))
missing_count.plot(kind='bar', color='steelblue')
plt.title('Jumlah Missing Value per Kolom - DataCampuran (Sebelum Dibuat Buatan)', fontsize=11)
plt.xlabel('Kolom')
plt.ylabel('Jumlah Missing')
plt.ylim(0, 3)
plt.tight_layout()
plt.show()
```



5. Membuat Missing Value Buatan

Missing value dibuat pada:

- **Baris indeks 2 (Citra)**
- **Kolom** `Tinggi`

```
target_idx = 2  # Citra
col_missing = 'Tinggi'
nilai_asli = df.loc[target_idx, col_missing]

print(f"Data asli baris {target_idx} ({df.loc[target_idx, 'Nama']}):")
print(df.loc[[target_idx]])
print(f"\nNilai asli {col_missing}: {nilai_asli}")

# Buat copy dan hilangkan nilai
df_knn = df.copy()
df_knn.loc[target_idx, col_missing] = np.nan

print(f"\nData baris {target_idx} setelah dibuat missing:")
print(df_knn.loc[[target_idx]])
```

```
Data asli baris 2 (Citra):
   No.   Nama  Umur  Tinggi Status Mahasiswa Jenis Kelamin
2    3  Citra    20     158               Ya             P

Nilai asli Tinggi: 158

Data baris 2 setelah dibuat missing:
   No.   Nama  Umur  Tinggi Status Mahasiswa Jenis Kelamin
2    3  Citra    20     NaN               Ya             P
```

```
# Tabel baris yang memiliki missing value
kolom_tampil = ['Nama', 'Umur', 'Tinggi', 'Status Mahasiswa', 'Jenis Kelamin']

fig, ax = plt.subplots(figsize=(10, 2))
ax.axis('off')
tabel = ax.table(
    cellText=df_knn.loc[[target_idx], kolom_tampil].values,
    colLabels=kolom_tampil,
    loc='center'
)
tabel.auto_set_font_size(False)
tabel.set_fontsize(9)
tabel.scale(1, 1.5)
plt.title('Baris Citra dengan Missing Value Buatan (Tinggi = NaN)', fontsize=11)
plt.tight_layout()
plt.show()
```

Baris Citra dengan Missing Value Buatan (Tinggi = NaN)

Nama	Umur	Tinggi	Status Mahasiswa	Jenis Kelamin
Citra	20	nan	Ya	P

6. Perhitungan Jarak Campuran (Manual + Kode)

Konsep

Karena data memiliki kolom numerik dan kategorikal:

1. **Numerik (`Umur`)** → Normalisasi Min-Max, lalu Euclidean Distance
2. **Kategorikal (`Status Mahasiswa`, `Jenis Kelamin`)** → $d_{kat} = \frac{P-M}{P}$
3. **Jarak Total** → $d_{total} = d_{num} + d_{kat}$

Normalisasi Umur (Min-Max)

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

`Umur_min = 19`, `Umur_max = 23`

Citra: $Umur'_{Citra} = \frac{20-19}{23-19} = 0.25$

```
# === PERHITUNGAN JARAK KNN - DATA CAMPURAN ===

# Normalisasi Umur (Min-Max)
umur_min = df_knn['Umur'].min()
umur_max = df_knn['Umur'].max()

def umur_norm(val):
    return (val - umur_min) / (umur_max - umur_min)

print(f"Umur_min = {umur_min}, Umur_max = {umur_max}")
print(f"Umur Citra = {df_knn.loc[target_idx, 'Umur']} -> normalized: {umur_norm(df_knn.loc[target_idx, 'Umur']):.4f}")

# Kolom kategorikal
cat_cols = ['Status Mahasiswa', 'Jenis Kelamin']

# Hitung jarak ke semua baris lain
hasil_jarak = []

for i in range(len(df_knn)):
    if i == target_idx:
        continue

    # Jarak numerik (Euclidean dari Umur saja, karena Tinggi = missing)
    d_num = np.sqrt(
        (umur_norm(df_knn.loc[target_idx, 'Umur']) - umur_norm(df_knn.loc[i, 'Umur']))**2
    )

    # Jarak kategorikal
    P = len(cat_cols)
    M = sum(df_knn.loc[target_idx, c] == df_knn.loc[i, c] for c in cat_cols)
    d_cat = (P - M) / P

    # Jarak total
    d_total = d_num + d_cat

    tinggi = df.loc[i, col_missing]  # ambil dari df asli
    nama = df.loc[i, 'Nama']
    hasil_jarak.append((i, nama, d_total, d_num, d_cat, tinggi))

# Urutkan berdasarkan jarak terkecil
hasil_jarak = sorted(hasil_jarak, key=lambda x: x[2])

# Tampilkan semua jarak
print("\n" + "=" * 60)
print("JARAK DARI CITRA KE SEMUA BARIS LAIN")
print("=" * 60)
```

```
for t in hasil_jarak:
    print(f"  {t[1]:6s}: d_total={t[2]:.4f} (d_num={t[3]:.4f}, d_kat={t[4]:.4f}), Tinggi={t[5]}")
```

```
Umur_min = 19, Umur_max = 23
Umur Citra = 20 -> normalized: 0.2500

============================================================
JARAK DARI CITRA KE SEMUA BARIS LAIN
============================================================
  Nana  : d_total=0.2500 (d_num=0.2500, d_kat=0.0000), Tinggi=160
  Andi  : d_total=0.7500 (d_num=0.2500, d_kat=0.5000), Tinggi=165
  Budi  : d_total=0.7500 (d_num=0.2500, d_kat=0.5000), Tinggi=170
  Dika  : d_total=1.5000 (d_num=0.5000, d_kat=1.0000), Tinggi=172
  Rudi  : d_total=1.7500 (d_num=0.7500, d_kat=1.0000), Tinggi=175
```

Verifikasi Manual

Citra vs Nana:

- Umur: $(0.25 - 0)^2 = 0.0625 \rightarrow d_{num} = 0.25$
- Status: Ya=Ya ✓, JK: P=P ✓ $\rightarrow d_{kat} = 0/2 = 0$
- $d_{total} = 0.25 + 0 = 0.25$

Citra vs Andi:

- Umur: $(0.25 - 0)^2 = 0.0625 \rightarrow d_{num} = 0.25$
- Status: Ya=Ya ✓, JK: P≠L ✗ $\rightarrow d_{kat} = 1/2 = 0.5$
- $d_{total} = 0.25 + 0.5 = 0.75$

Citra vs Budi:

- Umur: $(0.25 - 0.5)^2 = 0.0625 \rightarrow d_{num} = 0.25$
- Status: Ya=Ya ✓, JK: P≠L ✗ $\rightarrow d_{kat} = 0.5$
- $d_{total} = 0.25 + 0.5 = 0.75$

---

7. Menentukan 3 Tetangga Terdekat & Imputasi

```
# Ambil 3 tetangga terdekat
k = 3
tetangga = hasil_jarak[:k]

# Imputasi dengan rata-rata (KNN Regression)
imputasi = np.mean([x[5] for x in tetangga])

print("=" * 60)
print("IMPUTASI KNN - DATA CAMPURAN")
print("=" * 60)
print(f"Baris target: {target_idx} ({df.loc[target_idx, 'Nama']})")
print(f"Kolom missing: {col_missing}")
print(f"Nilai asli: {nilai_asli}")
print(f"\n3 tetangga terdekat:")
for t in tetangga:
    r = df.loc[t[0]]
    print(f"  {t[1]:6s}: d_total={t[2]:.4f} (d_num={t[3]:.4f}, d_kat={t[4]:.4f}), Tinggi={t[5]}")
    print(f"    Umur={r['Umur']}, Status={r['Status Mahasiswa']}, JK={r['Jenis Kelamin']}")
print(f"\nHasil imputasi (rata-rata): {imputasi:.4f}")
print(f"Nilai asli sebelum dihilangkan: {nilai_asli}")
```

```
============================================================
IMPUTASI KNN - DATA CAMPURAN
============================================================
Baris target: 2 (Citra)
Kolom missing: Tinggi
Nilai asli: 158

3 tetangga terdekat:
  Nana  : d_total=0.2500 (d_num=0.2500, d_kat=0.0000), Tinggi=160
    Umur=19, Status=Ya, JK=P
  Andi  : d_total=0.7500 (d_num=0.2500, d_kat=0.5000), Tinggi=165
    Umur=19, Status=Ya, JK=L
  Budi  : d_total=0.7500 (d_num=0.2500, d_kat=0.5000), Tinggi=170
    Umur=21, Status=Ya, JK=L

Hasil imputasi (rata-rata): 165.0000
Nilai asli sebelum dihilangkan: 158
```

Hasil

3 tetangga terdekat:

Peringkat	Nama	d_{num}	d_{kat}	d_{total}	Tinggi
1	Nana	0.25	0	**0.25**	160
2	Andi	0.25	0.5	**0.75**	165
3	Budi	0.25	0.5	**0.75**	170

Imputasi:

$$\hat{Tinggi} = \frac{160 + 165 + 170}{3} = \frac{495}{3} = 165.0$$

Nilai asli sebelum dihilangkan: **158**

---

8. Visualisasi Tabel Jarak

```
# Tabel semua jarak (5 baris selain Citra)
data_tabel = []
for t in hasil_jarak:
    data_tabel.append([t[1], f"{t[3]:.4f}", f"{t[4]:.4f}", f"{t[2]:.4f}", t[5]])

fig, ax = plt.subplots(figsize=(10, 4))
ax.axis('off')
tabel = ax.table(
    cellText=data_tabel,
    colLabels=['Nama', 'd_num', 'd_kat', 'd_total', 'Tinggi'],
    loc='center'
)
tabel.auto_set_font_size(False)
tabel.set_fontsize(10)
tabel.scale(1, 1.5)

# Warnai 3 baris teratas (tetangga terdekat)
for row in range(1, 4):
    for col in range(5):
        tabel[row, col].set_facecolor('#d4edda')

plt.title('Semua Jarak dari Citra — 3 hijau = tetangga terpilih (k=3)', fontsize=11)
plt.tight_layout()
plt.show()
```

Semua Jarak dari Citra — 3 hijau = tetangga terpilih (k=3)

Nama	d_num	d_kat	d_total	Tinggi
Nana	0.2500	0.0000	0.2500	160
Andi	0.2500	0.5000	0.7500	165
Budi	0.2500	0.5000	0.7500	170
Dika	0.5000	1.0000	1.5000	172
Rudi	0.7500	1.0000	1.7500	175

---

9. Hasil Akhir — Setelah Imputasi

```
# Isi missing value
df_hasil = df_knn.copy()
df_hasil.loc[target_idx, 'Tinggi'] = round(imputasi, 4)

print("Dataset setelah imputasi KNN:")
df_hasil[kolom_tampil]
```

```
Dataset setelah imputasi KNN:
```

	Nama	Umur	Tinggi	Status Mahasiswa	Jenis Kelamin
0	Andi	19	165.0	Ya	L
1	Budi	21	170.0	Ya	L
2	Citra	20	165.0	Ya	P
3	Dika	22	172.0	Tidak	L
4	Nana	19	160.0	Ya	P
5	Rudi	23	175.0	Tidak	L

```
# Tabel hasil imputasi baris Citra
fig, ax = plt.subplots(figsize=(10, 2))
ax.axis('off')
tabel = ax.table(
    cellText=df_hasil.loc[[target_idx], kolom_tampil].values,
    colLabels=kolom_tampil,
    loc='center'
)
tabel.auto_set_font_size(False)
tabel.set_fontsize(10)
tabel.scale(1, 1.5)
plt.title(f'Baris Citra Setelah Imputasi KNN (Tinggi = {imputasi:.1f})', fontsize=11)
plt.tight_layout()
plt.show()
```

Baris Citra Setelah Imputasi KNN (Tinggi = 165.0)

Nama	Umur	Tinggi	Status Mahasiswa	Jenis Kelamin
Citra	20	165.0	Ya	P

10. Perbandingan Sebelum & Sesudah

```
print("=" * 50)
print("PERBANDINGAN SEBELUM & SESUDAH IMPUTASI")
print("=" * 50)
print(f"Kolom       : {col_missing}")
print(f"Baris       : {target_idx} (Citra)")
print(f"Nilai asli  : {nilai_asli}")
print(f"Setelah KNN : {imputasi:.1f}")
print(f"Selisih     : {abs(nilai_asli - imputasi):.1f}")
print(f"\nMetode: KNN Regression (k=3)")
print(f"Jarak: Campuran (Euclidean numerik + kategorikal)")
```

```
==================================================
PERBANDINGAN SEBELUM & SESUDAH IMPUTASI
==================================================
Kolom       : Tinggi
Baris       : 2 (Citra)
Nilai asli  : 158
Setelah KNN : 165.0
Selisih     : 7.0

Metode: KNN Regression (k=3)
Jarak: Campuran (Euclidean numerik + kategorikal)
```

Ringkasan

Langkah KNN Imputation pada Data Campuran:

1. Tentukan tipe setiap kolom: numerik atau kategorikal
2. Normalisasi kolom numerik dengan Min-Max Scaling
3. Hitung jarak numerik (Euclidean)
4. Hitung jarak kategorikal: $(P - M)/P$
5. Jumlahkan: $d_{total} = d_{num} + d_{kat}$
6. Ambil k=3 tetangga terdekat
7. Isi missing value = rata-rata dari 3 tetangga (KNN Regression)